

Homomorphic Encryption

Definitions and First Examples

SEC 595 — Encrypted Computing

- 1 Homomorphic Encryption
- 2 Partially Homomorphic Encryption
- 3 A Scheme With Both Operations

An HE scheme is four algorithms

Definition

- $\text{KeyGen}(\lambda) \rightarrow (\text{Pk}, \text{Sk})$
- $\text{Enc}(\text{Pk}, \text{pt}) \rightarrow \text{ct}$
- $\text{Dec}(\text{Sk}, \text{ct}) \rightarrow \text{pt}$
- $\text{Evaluate}(\text{Pk}, \Pi, \text{ct}_1, \dots, \text{ct}_k) \rightarrow \hat{\text{ct}}$

The first three are ordinary public-key encryption — Regev has them.

Evaluate applies a function Π to ciphertexts. It takes **neither the secret key nor the plaintexts**: a server can run it without seeing the data.

Correct for Π

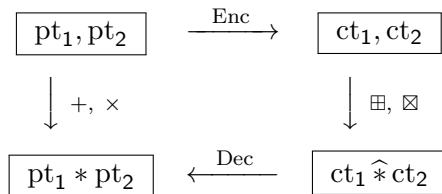
- 1 $\text{Dec}(\text{Sk}, \text{Enc}(\text{Pk}, \text{pt})) = \text{pt}$
- 2 $\text{Dec}(\text{Sk}, \text{Evaluate}(\text{Pk}, \Pi, \text{ct}_1, \dots, \text{ct}_k)) = \Pi(\text{pt}_1, \dots, \text{pt}_k)$

Property 2 is the homomorphism: compute-then-decrypt equals decrypt-then-compute.

“Homomorphic” always means homomorphic *for some* Π .

Lattice schemes achieve both with probability $1 - \text{negl}(\lambda)$, not 1.

The commuting square



$\boxplus$ and $\boxtimes$ are whatever the scheme supplies — rarely the same as $+$ and $\times$.

For a deterministic Enc: $\text{Enc}(pt_1) \hat{*} \text{Enc}(pt_2) = \text{Enc}(pt_1 * pt_2)$. For a randomised one, equality holds only after decryption.

Compactness

A cheat: let `Evaluate` return $(\Pi, ct_1, \dots, ct_k)$, and let `Dec` decrypt the inputs and apply Π . Both correctness properties hold — and the server computed nothing.

Compactness

The output of `Evaluate` has size bounded by a polynomial in λ , independent of Π and of the number of inputs.

The cost of `Dec` does not grow with the work `Evaluate` did. Compactness is about *size*, not *speed*.

Attacker (holds Pk)		Challenger
choose m_0, m_1	$\longrightarrow$	random bit b
guess b'	$\longleftarrow$	$ct^* = \text{Enc}(m_b)$

IND-CPA (semantic security)

For every efficient attacker, $\left| \Pr[b' = b] - \frac{1}{2} \right| \leq \text{negl}(\lambda)$.

The ciphertext reveals nothing — not even which of two known messages it encrypts.

Two consequences for HE

- 1 **Encryption must be randomised.** If Enc is deterministic, the attacker computes $\text{Enc}(m_0)$ itself and compares with ct^* .
- 2 **HE cannot be CCA2 secure.** In CCA2 the attacker may decrypt anything except ct^* . With Evaluate it forms $\text{ct}^* \boxtimes \text{Enc}(1)$ — a different ciphertext of the same plaintext — has it decrypted, and reads off m_b .

So homomorphic schemes aim for IND-CPA.

	Operations	How many
PHE — partially	one (+ or $\times$)	unlimited
SWHE — somewhat	both	limited
LHE — levelled	both	depth L , fixed at KeyGen
FHE — fully	both	unlimited

Over $\mathbb{Z}_2$, $+$ is XOR and $\times$ is AND. Every Boolean circuit is built from these, so FHE can evaluate *any* function.

PHE restricts *which*; SWHE restricts *how many*.

$$n = pq, \gcd(e, (p-1)(q-1)) = 1, d = e^{-1} \bmod (p-1)(q-1).$$

$$\text{Pk} = (e, n), \quad \text{Sk} = d,$$

$$\text{Enc}(x) = x^e \bmod n, \quad \text{Dec}(\text{ct}) = \text{ct}^d \bmod n.$$

Multiplicatively homomorphic

$$\text{Enc}(x) \text{Enc}(y) \equiv x^e y^e = (xy)^e = \text{Enc}(xy) \pmod{n}$$

No operation gives $\text{Enc}(x + y)$: $(x + y)^e$ has cross terms $x^j y^{e-j}$ that cannot be formed from x^e and y^e .

Example — RSA by hand

$p = 11$, $q = 13$, $e = 7$.

- a. Compute n and d .
- b. Encrypt $x = 5$ and $y = 7$.
- c. Multiply the ciphertexts and decrypt.
- d. Add the ciphertexts instead and decrypt. What do you get?

RSA is not IND-CPA secure

Enc is deterministic, so the attacker encrypts m_0 itself and compares with ct^* . It wins every time.

Deployed RSA pads the message with randomness — which also destroys the homomorphism.

The determinism that makes $\text{Enc}(x)\text{Enc}(y) = \text{Enc}(xy)$ exact is the determinism that breaks IND-CPA.

P prime, g a primitive root modulo P .

Scheme

- KeyGen: random X , $y = g^X \bmod P$. $\text{Pk} = (P, g, y)$, $\text{Sk} = X$.
- Enc(m): random k , output $(c_1, c_2) = (g^k, m y^k) \bmod P$.
- Dec: $s = c_1^X$, output $m = c_2 s^{-1} \bmod P$.

Correct because $s = (g^k)^X = (g^X)^k = y^k$, the mask.

Randomised by k ; IND-CPA secure under DDH.

Multiply componentwise:

$$(g^{k_1}, m_1 y^{k_1}) \cdot (g^{k_2}, m_2 y^{k_2}) = (g^{k_1+k_2}, (m_1 m_2) y^{k_1+k_2}).$$

The result is an encryption of $m_1 m_2$ with randomness $k_1 + k_2$.

Two schemes, two hard problems — factoring and discrete log — and the same missing addition.

Example — ElGamal by hand

$$P = 23, g = 5, X = 6.$$

- a. Compute y .
- b. Encrypt $m = 10$ with $k = 10$, and decrypt.
- c. Encrypt $m_1 = 3$ with $k_1 = 10$ and $m_2 = 4$ with $k_2 = 3$. Multiply the ciphertexts and decrypt.

KeyGen

① Primes p, q with $\gcd(pq, (p-1)(q-1)) = 1$; $n = pq$.

② $\lambda = \text{lcm}(p-1, q-1)$.

③ Choose $g \in \mathbb{Z}_{n^2}^*$.

④ $\mu = L(g^\lambda \bmod n^2)^{-1} \bmod n$, where $L(x) = \frac{x-1}{n}$.

$\text{Pk} = (n, g)$, $\text{Sk} = (\lambda, \mu)$.

L is integer division, applied only to $x \equiv 1 \pmod{n}$.

The inverse μ must exist; $g = n + 1$ always works, with $\mu = \lambda^{-1} \bmod n$.

Scheme

$$\text{Enc}(m) = g^m r^n \bmod n^2, \quad r \in \mathbb{Z}_n^* \text{ random}, \quad 0 \leq m < n$$

$$\text{Dec}(c) = L(c^\lambda \bmod n^2) \cdot \mu \bmod n$$

The message sits in the *exponent*; r^n randomises.

Raising to λ kills the randomiser ($r^{n\lambda} \equiv 1 \bmod n^2$); L reads off m times a constant; μ cancels the constant.

$$c_1 c_2 = g^{m_1} r_1^n g^{m_2} r_2^n = g^{m_1+m_2} (r_1 r_2)^n = \text{Enc}(m_1 + m_2 \bmod n)$$

Multiplying ciphertexts *adds* plaintexts: here $\boxtimes$ is $\times$.

A public constant k : c^k decrypts to $km \bmod n$.

Two encrypted values cannot be multiplied — Paillier is PHE for $+$.

Example — Paillier by hand

$p = 7$, $q = 11$, $g = 5$.

- a. Compute n , λ and μ .
- b. Encrypt $m_1 = 15$ with $r = 23$, and decrypt.
- c. Encrypt $m_2 = 20$ with $r = 30$. Multiply the ciphertexts and decrypt.
- d. What does the product of encryptions of 70 and 20 decrypt to?

van Dijk, Gentry, Halevi, Vaikuntanathan (2010).

Scheme

- KeyGen: a large odd integer p .
- Enc(m), $m \in \{0, 1\}$: random large q , random small r ,

$$c = pq + 2r + m.$$

- Dec(c): $(c \bmod p) \bmod 2$, with $c \bmod p$ centred in $(-p/2, p/2]$.

The **noise** is $2r + m$. Decryption is correct while $|2r + m| < p/2$.

Security: the approximate-GCD problem.

$$c_1 + c_2 = p(q_1 + q_2) + 2(r_1 + r_2) + (m_1 + m_2)$$

$$c_1 c_2 = p(\dots) + 2(2r_1 r_2 + r_1 m_2 + r_2 m_1) + m_1 m_2$$

So $c_1 + c_2$ encrypts $m_1 \oplus m_2$, and $c_1 c_2$ encrypts $m_1 \wedge m_2$ — while the noise stays below $p/2$.

Addition adds noise. Multiplication multiplies it.

Example — the integer scheme by hand

$p = 101$. Encrypt $m_1 = 1$ with $q = 7$, $r = 2$, and $m_2 = 1$ with $q = 5$, $r = 3$.

- a. Compute c_1 , c_2 and their noises; decrypt.
- b. Decrypt $c_1 + c_2$ and $c_1 c_2$.
- c. Decrypt c_2^2 and c_2^3 . Which one fails, and why?

Noise grows with every multiplication, so the *depth* is limited: the scheme is **SWHE**. Sizing p for a fixed depth makes it **LHE**.

Bootstrapping (Gentry, 2009)

If a scheme can evaluate its own decryption circuit, with room for one more operation, a noisy ciphertext can be refreshed: evaluate Dec under encryption, using an encryption of the secret key.

The result encrypts the same bit with fresh noise. SWHE plus bootstrapping is **FHE**.

- ① HE = KeyGen, Enc, Dec, Evaluate; Evaluate uses neither the secret key nor the plaintexts.
- ② Required: correctness for Π , compactness, IND-CPA (so randomised encryption). HE cannot be CCA2 secure.
- ③ PHE: one operation, unlimited. SWHE: both, limited. LHE: both, fixed depth. FHE: both, unlimited.
- ④ RSA and ElGamal: multiplicative PHE. Paillier: additive PHE.
- ⑤ $c = pq + 2r + m$ adds and multiplies bits until the noise reaches $p/2$; bootstrapping removes the limit.